PowerShell + DevOps
Global Summit

POWERSHELL
DEVOPS 2026
GLOBAL SUMMIT

STOP HAND-ROLLING CHOCOLATE

AUTOMATING CHOCOLATEY WITH PSAKE

Gilbert Sanchez

@HeyItsGilbert

April 13-16, 2026

POWERSHELL DEVOPS 2026 GLOBAL SUMMIT

Thanks

Jane Street

fleet

PATCH MY PC

PDQ

SynEdgy

Hey! It's Gilbert

- Staff Software Development Engineer
- ADHD 🌶️🧠
- Links.GilbertSanchez.com

Hands Up

”Back in my day...”

The year is 2013

The Build Process

"The ticket queue was the build process."

- Teams needed a Windows packages built and deployed.
- They filed a ticket
- Someone (eventually) hand-crafted it
- That someone was the only one who knew how

”It Works on My Machine”

Except your devs are on Macs

- File locking differently
- Windows-specific paths, encodings, behaviors
- Packaging Windows software felt like a **foreign language**

Extra friction became an extra excuse.

Chocolatey is just PowerShell

If you can write a script, you can ship a package.

Scaffolding a Package

```
choco new mypackage
```

```
mypackage/
├── mypackage.nuspec           ← package metadata
└── tools/
    ├── chocolateyInstall.ps1  ← runs on install
    ├── chocolateyUninstall.ps1
    └── LICENSE.txt
```

That's the whole package.

The Two Files That Matter: Metadata

mypackage.nuspec — *what* the package is

```
<metadata>
  <id>mypackage</id>
  <version>1.0.0</version>
  <description>Does a thing.</description>
</metadata>
```

The Two Files That Matter: Installer

tools/chocolateyInstall.ps1 — *how* it installs

```
$packageArgs = @{
  packageName = 'mypackage'
  url         = 'https://example.com/installer.exe'
  checksum    = 'ABC123...'
}
Install-ChocolateyPackage @packageArgs
```

Extensions

Add new functions to any package

```
mycompany-chocolatey-extension/
└── extensions/
    └── mycompany-helpers.psm1  ← imported automatically
```

Hooks

Run logic around every install

```
mycompany-hooks/
└── hooks/
    ├── pre-install-all.ps1     ← before any package installs
    └── post-install-all.ps1    ← after any package installs
```

psake

Tasks with dependencies

That's it. PowerShell Tasks.

Your Build Process is a README and Hope

```
# HOW TO RELEASE (don't forget these steps)
1. Validate the nuspec manually
2. Run choco pack
3. Run the tests (yes, before you push JOHN!)
4. Push to the feed — but only on main!
```

Every step is a chance for someone to skip it.

Manual Steps → Declared Tasks

```
Task Default -depends Pack

Task Test -depends -description "Run Pester tests" {
    Invoke-Pester .\Tests\
}

Task Pack -depends Test -description "Build the .nupkg" {
    exec { choco pack }
}

Task Push -depends Pack -precondition { $env:GITHUB_REF -eq 'refs/heads/main' } {
    exec { choco push mypackage.nupkg --source $env:CHOCO_FEED }
}
```

The Task Graph

```
Push  (only on main)
 └── Pack
      └── Test
```

Run everything: `Invoke-psake`

Run just tests: `Invoke-psake -taskList Test`

Same command. Local or CI. No surprises.

The Brazil Story

The Brazil Story

Extensions put your org's knowledge in one place.

```powershell
# mycompany-chocolatey-extension/extensions/mycompany-helpers.psm1
function Get-PackageFromCDN {
    param($PackageName, $Version)

    $node   = Resolve-NearestCDNNode          # ← finds São Paulo, not Virginia
    $cached = Test-CDNCache -Node $node -Package $PackageName

    if ($cached) { Get-FromCache  -Node $node -Package $PackageName }
    else         { Get-FromOrigin -Package $PackageName -Version $Version }
}
```

Every package calls `Get-PackageFromCDN`*. Nobody hard-codes the CDN.*

Let’s build it

Blank repo → published, tested, CI-backed package.

What We Just Built

STEP	WHAT IT DOES
`.\build.ps1 -Task Test`	Runs Pester tests against the package
`.\build.ps1 -Task Pack`	Builds the `.nupkg`
`.\build.ps1 -Task Push`	Publishes — but only from main
CI workflow	Validates + tests every PR
Publish workflow	Auto-publishes on merge

Same psakefile. Local and CI. No surprises.

”I Only Have 5 Packages”

You still get:

- ✓ Auto-publish on merge
- ✓ Validated nuspec XML
- ✓ Pester tests
- ✓ Extensions & Hooks

Automation that reduces toil scales down just as well as it scales up.

Your Mac Dev Just Shipped a Windows Package

They didn't touch a Windows machine.

They didn't learn Chocolatey internals.

They opened a PR. CI went green. They merged.

The packaging knowledge lives in the psakefile — not in someone's head.

Boring is good

Predictable. Auditable. Repeatable.

The best build pipeline is the one you forget about.

PowerShell + DevOps
Global Summit

POWERSHELL
DEVOPS 2026
GLOBAL SUMMIT

THANK YOU

FEEDBACK IS A GIFT

Please review this session via the mobile app

Questions? Find me @heyitsgilbert

April 13-16, 2026